A-PARTMENT HELPER

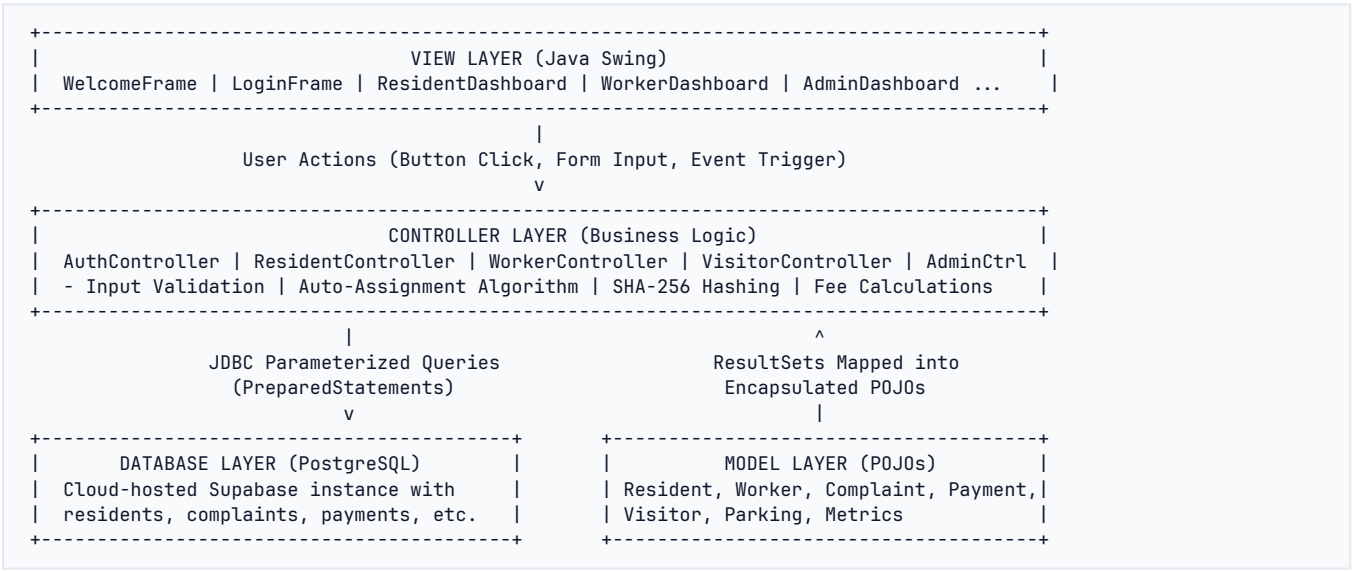
Comprehensive Codebase Explanation, Java Syllabus Reference & Viva Guide

TECH STACK: Java 17/21 • Java Swing (MVC GUI) • PostgreSQL / Supabase • Academic Modules 1 – 4

PART I: SYSTEM OVERVIEW & ARCHITECTURE

The Model-View-Controller design pattern and system communication flow

The A-partment Helper system is an enterprise-grade desktop society management application engineered in Java using the Model-View-Controller (MVC) architectural pattern. MVC cleanly separates visual user presentation from application business rules, database queries, and data models.



Architectural Benefits of MVC:

- **Decoupled Presentation:** Swing View components contain no database connection strings or SQL queries. Views strictly collect user input and render visual components.
- **Maintainable Business Logic:** Controller classes encapsulate business decisions, validation, and auto-assignment algorithms. Updating SQL queries or validation rules does not affect the UI design.
- **Data Integrity & Encapsulation:** Model classes (POJOs) encapsulate database records with private fields and public accessors, preventing unauthorized side effects.

PART II: COMPLETE FILE-BY-FILE CODEBASE EXPLANATION (ALL 24 FILES)

Detailed breakdown of the role, responsibilities, and mechanics of every source file

1. Entry Point & Core Configuration

- **src/Main.java:** The application bootstrapper. Configures OS-level font anti-aliasing via System.setProperty('awt.useSystemAAFontSettings', 'on') and 'swing.aatext' for sharp typography. Safely initializes WelcomeFrame on the Swing Event Dispatch Thread (EDT) using SwingUtilities.invokeLater().

2. Database & Utility Classes

- **src/database/DBConnection.java:** Centralized database factory. Dynamically loads the PostgreSQL JDBC driver ('org.postgresql.Driver'), securely encapsulates Supabase credentials (URL, USER, PASSWORD), and provides the getConnection() factory method returning live java.sql.Connection instances.
- **src/util/CodeGenerator.java:** Cryptographic security utility. Generates 4-character random visitor pass codes (e.g., 'K7M2') and 3-character checkout codes ('7B2') using SecureRandom. Implements sha256(String) using Java's MessageDigest to hash pass codes so plaintext codes are never stored in the database.

3. Model Layer (src/model/ POJOs)

- **src/model/Resident.java:** Encapsulates apartment resident data: residentId, fullName, email, phone, apartmentNo, parkingSlot, and account status.
- **src/model/Worker.java:** Encapsulates maintenance staff: workerId, fullName, workerType (Plumbing, Electrical, Carpentry, Cleaning), phone, and activeComplaintsCount for workload balancing.

- **src/model/Admin.java:** Encapsulates administrative accounts with credentials, email, and administrative role privileges.
- **src/model/Complaint.java:** Encapsulates maintenance tickets: complaintId, residentId, assignedWorkerId, title, category, description, status (PENDING, ASSIGNED, IN_PROGRESS, COMPLETED), and resolutionNote.
- **src/model/Payment.java:** Encapsulates society billing and invoices: paymentId, residentId, title, amount, status (PENDING, PAID), dueDate, paidDate, and paymentMethod.
- **src/model/Visitor.java:** Encapsulates visitor passes: visitorId, residentId, name, phone, hasVehicle, parkingSlot, accessHash (SHA-256), exitCode, and entry/exit timestamps.
- **src/model/Parking.java:** Encapsulates physical parking bays: parkingId, slotNumber (R-xx for residents, V-xx for visitors), slotType, status (AVAILABLE, OCCUPIED), assigned residentId, or visitorId.
- **src/model/ResidentMetrics.java:** Aggregates summary statistics for a resident's dashboard (pending bills count, outstanding balance, open complaints, visitors today).
- **src/model/SocietyMetrics.java:** Aggregates macro KPIs for the administrative dashboard (total residents, active staff, open complaints, monthly collections, occupied parking).

4. Controller Layer (src/controller/ Business Logic & JDBC)

- **src/controller/AuthController.java:** Authenticates login credentials across residents, workers, and admins. Enforces password checks, handles session routing to appropriate dashboard windows, and manages clean logout.
- **src/controller/ResidentController.java:** Coordinates resident operations: loads resident profile, aggregates dashboard metrics, creates visitor passes with SHA-256 hashing, registers complaints with auto-worker assignment, and processes bill payments.
- **src/controller/WorkerController.java:** Coordinates technician workflows: retrieves assigned and category-specific open tickets, claims unassigned complaints, updates status to IN_PROGRESS, records completion notes, and generates automated repair fee invoices.
- **src/controller/VisitorController.java:** Coordinates security gate check-in/out: verifies pass codes by hashing and matching access_hash, dynamically allocates visitor parking slots (V-01 to V-10), generates exit codes, and deallocates parking slots upon exit.
- **src/controller/AdminController.java:** Coordinates society administration: computes society-wide KPIs, executes full CRUD for residents and staff, synchronizes assigned resident parking slots, manually or automatically assigns complaints, and generates custom resident invoices.

5. View Layer (src/view/ Java Swing User Interface)

- **src/view/WelcomeFrame.java:** The landing window with modern buttons directing users to either 'Login to Portal' or 'Visitor Entry / Exit' (security desk).
- **src/view/LoginFrame.java:** Authentication window featuring a role selector (RESIDENT, WORKER, ADMIN), email, and password fields, delegating validation to AuthController.
- **src/view/ResidentDashboard.java:** Multi-tab resident portal: Overview (KPI cards, quick actions), Visitors (invitation modal, past visitors), Complaints (ticket submission form, live status), Bills (unpaid bills with 'Pay Now' flow).
- **src/view/WorkerDashboard.java:** Technician workbench: active jobs queue, ticket claiming, status transitions (IN_PROGRESS, COMPLETED), repair note entry, and billing dialog.
- **src/view/AdminDashboard.java:** Administrative control panel: society overview KPIs, resident management with Add/Edit/Delete modals, worker workload balancer, complaint triage with auto-assign, and financial ledger.
- **src/view/VisitorEntryFrame.java:** Security guard workstation with two tabs: Gate Check-In (validates 4-character pass, displays allocated parking slot and exit code) and Gate Check-Out (releases visitor and parking slot).
- **src/view/InviteVisitorFrame.java:** Modal dialog launched from ResidentDashboard allowing residents to register a guest, select vehicle requirements, and display the generated 4-character pass code.

PART III: COMPREHENSIVE JAVA SYLLABUS & ACADEMIC CONCEPT MAPPING

Module-by-Module mapping (Modules 1 – 4) across Core Java, OOP, Design Patterns & JDBC

Module 1: Introduction to Java & OOP Concepts

1. Structure of a Simple Java Program & Execution Environment

- **Entry Point:** Main.java defines public static void main(String[] args), the standardized JVM entry point.
- **Compiler & Bytecode:** Source code is compiled by javac into architecture-neutral .class bytecode files in out/.
- **Java Virtual Machine (JVM):** The JVM interprets bytecode using java -cp 'lib/*:out' Main, resolving classpath dependencies like lib/postgresql-42.7.3.jar.

2. Primitive Data Types vs. Wrapper Types, Autoboxing & Casting

- **Primitive Types:** int for entity IDs and counters; double for financial amounts (Payment.amount); boolean for flags (hasVehicle); char and byte[] for SHA-256 byte digests in CodeGenerator.
- **Wrapper Types & Nullability:** Integer and Double wrappers are utilized for nullable database foreign keys: Integer workerId in Complaint.java (null when unassigned), Integer residentId in Parking.java, and Double repairFee in WorkerController.java.

- **Autoboxing & Unboxing:** In ResidentController.java, SQL nullable columns are mapped using: Integer workerId = rs.getObject('worker_id') != null ? rs.getInt('worker_id') : null;
- **Type Casting:** Graphics2D g2 = (Graphics2D) g.create(); in ResidentDashboard.java for custom painting; (DefaultTableModel) table.getModel() for table data manipulation.

3. Arrays, Strings, Vector, Operators & Control Flow

- **Arrays:** Table column headers (String[] cols = {'ID', 'Title', 'Status'};) and table row arrays (model.addRow(new Object[]{...});).
- **Strings & StringBuilder:** StringBuilder used in CodeGenerator for thread-safe random code assembly; String.format('%.2f', amount) for currency; methods like isBlank(), trim(), equals().
- **Vector Class:** While modern collections (ArrayList) are used in controllers, Swing's DefaultTableModel internally uses java.util.Vector to manage rows and column names.
- **Control Flow:** Ternary operator (hasVehicle ? 'YES' : 'NO'); switch statements in AuthController for role dispatch; while (rs.next()) for database cursor traversal; enhanced for loops (for (Complaint c : list)).

Module 2: Polymorphism, Inheritance & Advanced Class Features

1. Polymorphism & Method Overloading

- **Compile-Time Polymorphism:** WorkerController overloads completeComplaint(): a 4-parameter version for standard closing, and a 6-parameter version that simultaneously records notes and generates a billing invoice.
- **Constructor Overloading:** Resident.java and Worker.java implement both default no-arg constructors (new Worker()) and fully parameterized constructors for instantiation flexibility.

2. Static Members, Final Variables & Inner Classes

- **Static Methods & State:** DBConnection.getConnection() and CodeGenerator.generateCode() are static utility methods callable without object instantiation. DBConnection holds static connection credentials.
- **Final Constants:** private static final String CHARS in CodeGenerator and private static final String URL in DBConnection enforce immutability.
- **Static Nested Classes:** AdminDashboard.WorkerItem and ResidentItem are static nested helper classes used in dropdown selectors.
- **Anonymous Inner Classes:** Mouse adapters for table click detection (table.addMouseListener(new MouseAdapter() { public void mouseClicked(...) { ... } })) and custom table renderers.

3. Inheritance, Method Overriding & Dynamic Dispatch

- **Superclass & Subclass Hierarchy:** javax.swing.JFrame is the Superclass. All 7 GUI views (AdminDashboard, ResidentDashboard, WorkerDashboard, LoginFrame, etc.) are Subclasses extending JFrame.
- **Method Overriding (@Override):** Worker.java and Resident.java override Object.toString() so JComboBox components display formatted names and IDs. Table cell renderers override getTableCellRendererComponent().
- **Dynamic Method Dispatch:** When Swing repaints components, it invokes overridden paintIcon() and render methods dynamically at runtime through base class/interface references.
- **Constructor Execution Order:** When new ResidentDashboard() is invoked, Java automatically executes the superclass JFrame constructor before executing ResidentDashboard constructor body.

Module 3: Packages, Interfaces, Exception Handling & Design Patterns

1. Package Architecture & CLASSPATH

- **Package Organization:** The codebase is cleanly separated into 5 packages: package model, package controller, package view, package database, and package util.
- **Access Protection:** public classes/methods for APIs; private fields for POJOs; protected methods inherited from JFrame.

2. Interfaces vs. Abstract Classes

- **Interface Implementation:** SwingNavIcon implements javax.swing.Icon. JDBC interfaces (Connection, PreparedStatement, ResultSet) in java.sql are interfaces whose implementations are supplied by the PostgreSQL driver.
- **Abstract Classes:** java.awt.Component and javax.swing.JComponent are root abstract classes for all UI elements. java.awt.event.MouseAdapter is an abstract adapter class.

3. Exception Handling Architecture

- **Checked Exceptions:** SQLException and ClassNotFoundException must be handled or declared. Handled via try-catch blocks and declared with throws SQLException in controller methods.
- **Unchecked Exceptions:** Validation logic explicitly throws IllegalArgumentException (e.g., duplicate email) or NumberFormatException (e.g., negative billing amount).
- **Try-with-Resources:** Used across every single JDBC method to automatically close Connection, PreparedStatement, and ResultSet instances, guaranteeing zero memory or connection pool leaks.

4. Design Patterns: Singleton & Adapter

- **Singleton Pattern:** DBConnection.java acts as a centralized connection provider managing static database parameters and connection lifecycles.
- **Adapter Pattern:** MouseAdapter adapts the 5-method MouseListener interface, allowing AdminDashboard to implement only mouseClicked(). SwingNavIcon adapts procedural 2D vector drawing to Swing's Icon interface.

Module 4: SOLID Principles, Swing GUI & JDBC

1. SOLID Principles in Practice

- **S - Single Responsibility:** model/ only stores state; controller/ only handles business logic and DB; view/ only renders GUI; DBConnection only manages connection lifecycles.
- **O - Open/Closed Principle:** New complaint categories (e.g., 'Elevator') or payment types can be added without modifying the core controller logic or database schema.
- **L - Liskov Substitution:** Any view subclassing JFrame can be substituted wherever a top-level Swing window is expected.
- **I - Interface Segregation:** Using MouseAdapter prevents classes from being forced to implement dummy methods for mouseEntered, mouseExited, etc.
- **D - Dependency Inversion:** Views depend on high-level controller abstractions (workerController.getAllComplaints()) rather than low-level JDBC query implementations.

2. Swing GUI Fundamentals & Layout Managers

- **AWT vs. Swing:** AWT provides underlying OS text anti-aliasing; lightweight Swing components (JFrame, JPanel, JButton, JTable, JScrollPane) provide flexible cross-platform UI.
- **Layout Managers Used:** BorderLayout for root frames; CardLayout for in-place tab switching without opening new OS windows; GridLayout for dashboard KPI cards; BoxLayout for vertical sidebars; FlowLayout for action button bars.

3. Event Handling & Delegation Event Model

- **Delegation Mechanism:** Event sources (JButton, JTable) generate events (ActionEvent, MouseEvent) that are delegated to registered listeners (ActionListener, MouseListener).
- **Lambda Listeners:** Modern concise lambda syntax: loginBtn.addActionListener(e -> handleLogin());
- **Thread Safety (EDT):** Application is launched safely on the Swing Event Dispatch Thread (EDT) via SwingUtilities.invokeLater() in Main.java.

4. Database Applications using JDBC

- **Full CRUD Operations:** Create: INSERT INTO residents, complaints; Read: SELECT with JOINs (joining complaints with residents and workers); Update: UPDATE complaints SET status = 'COMPLETED'; Delete: DELETE FROM residents WHERE resident_id = ?.
- **ResultSet Traversal:** rs.next() with strongly-typed accessors: rs.getInt(), rs.getString(), rs.getDouble(), rs.getTimestamp().
- **SQL Injection Prevention:** Exclusive use of PreparedStatement with parameterized placeholders (?) across every single controller method.

5. Quick-Reference OOP & Academic Concepts Summary Table

Concept	Definition & Academic Meaning	Concrete Project Implementation
Encapsulation	Data hiding with private fields + public getters/ setters.	All src/model/ POJOs; private credentials in DBConnection.java.
Inheritance	Deriving new classes from superclasses (extends).	All 7 GUI views extend JFrame; cell renderers extend DefaultTableCellRenderer.
Polymorphism (Overload)	Same method name with distinct parameter signatures.	WorkerController.completeComplaint() (4 vs 6 params); Model constructors.
Polymorphism (Override)	Redefining superclass behavior (@Override).	toString() in Worker & Resident; getTableCellRendererComponent() in tables.
Interface Polymorphism	Programming to abstract interface contracts.	List<T> = new ArrayList<>(); JDBC Connection, PreparedStatement, ResultSet.
Abstraction	Exposing clean APIs while hiding low-level complexity.	Controllers abstract SQL/DB from Views; CodeGenerator abstracts crypto hashing.
Singleton Pattern	Single global access point for resource management.	DBConnection provides centralized static connection factory getConnection().
Adapter Pattern	Converting incompatible interfaces into expected types.	MouseAdapter adapts MouseListener; SwingNavIcon adapts vector 2D drawing.

PART IV: END-TO-END WORKFLOWS & SECURITY LIFECYCLES

Detailed execution flows for authentication, visitor tracking, parking, complaints, and billing

1. Role-Based Authentication & Session Handoff
LoginFrame captures the user's email, password, and chosen role (RESIDENT, WORKER, ADMIN). AuthController.login() queries the respective table. Upon successful match, it instantiates the corresponding dashboard (passing authenticated ID) and disposes of the login frame. Invalid credentials trigger an error dialog.

2. Visitor Pass Generation & SHA-256 Hashing
When a resident invites a guest via InviteVisitorFrame, ResidentController.createVisitorPass() generates a 4-character random code (e.g. 'A9K2') via CodeGenerator.generateCode(). To prevent exposure in case of database leaks, the raw code is hashed using SHA-256 before being stored in visitors.access_hash. The plaintext code is displayed once to the resident to share with the guest.

3. Gate Check-In & Dynamic Parking Bay Allocation
At the gate, the guard enters the visitor's 4-character code into VisitorEntryFrame. VisitorController.processEntry() hashes the input with SHA-256 and searches for a PENDING record. If the visitor brings a vehicle, allocateVisitorParking() finds an available visitor parking bay (V-01 to V-10), marks it OCCUPIED, links it to the visitor, and generates a 3-character checkout code ('TEL'). Status transitions to ENTERED.

4. Gate Check-Out & Automatic Parking Bay Deallocation
Upon departure, the guard enters the 3-character exit code. VisitorController.processExit() locates the entered visitor, sets status to EXITED, records exit timestamp, and automatically resets the assigned visitor parking bay back to AVAILABLE.

5. Maintenance Complaint Lifecycle & Auto-Assignment Algorithm
The maintenance ticket lifecycle follows a strict state transition: PENDING -> ASSIGNED -> IN_PROGRESS -> COMPLETED.

- **Triage & Auto-Assignment:** When a resident submits a complaint (e.g. 'Plumbing'), ResidentController.fileComplaint() queries all active plumbing workers, counts their active tickets, and automatically assigns the technician with the lightest workload.
- **Execution & Status Update:** The technician opens WorkerDashboard, claims or starts work, transitioning status to IN_PROGRESS.
- **Completion & Billing:** Upon repair completion, the worker records notes and an optional repair fee. WorkerController.completeComplaint() marks status COMPLETED and automatically inserts an invoice into the payments table.

PART V: TOP VIVA & TECHNICAL INTERVIEW QUESTIONS

Concise, expert model answers for examinations and project defenses

Q1: What is JDBC and how does it connect to the PostgreSQL database?
Answer: JDBC (Java Database Connectivity) is a standard Java API that connects Java applications to relational databases. In DBConnection.java, we load the driver class 'org.postgresql.Driver' via Class.forName() and establish a secure SSL connection using DriverManager.getConnection(url, user, password).

Q2: Why did you use PreparedStatement instead of Statement?
Answer:
1. Security: PreparedStatement uses parameterized queries with '?' placeholders, completely preventing SQL Injection attacks by automatically escaping user inputs.
2. Performance: PreparedStatement is pre-compiled by the database engine, allowing faster repeated execution.

Q3: What is 'try-with-resources' and why is it important?
Answer: Introduced in Java 7, try-with-resources automatically closes any resource implementing AutoCloseable (Connection, PreparedStatement, ResultSet) when the block finishes, even if an exception is thrown. This guarantees zero database connection leaks.

Q4: How does visitor pass verification maintain security?
Answer: Plaintext pass codes are never stored in the database. When created, the pass code is hashed using SHA-256 (CodeGenerator.sha256()) and stored as access_hash. At the gate, the guard's input is hashed and compared against access_hash in the database.

Q5: How does the auto-assignment algorithm distribute maintenance work?
Answer: findBestWorkerForCategory() filters active workers matching the complaint category (e.g. Electrical), counts their current active tickets (ASSIGNED/IN_PROGRESS), and assigns the ticket to the technician with the lightest current workload.

Q6: How are Design Patterns applied in this project?
Answer:
1. Singleton Pattern: In DBConnection.java for centralized connection access.
2. Adapter Pattern: MouseAdapter simplifies mouse event handling; SwingNavIcon adapts vector drawing to the Icon interface.
3. MVC Pattern: Architectural pattern separating Model (data), View (GUI), and Controller (logic).

PART VI: VERIFICATION, BUILD & EXECUTION COMMANDS

Standard terminal commands for compilation, testing, and deployment

- **Compile All Source Code:** javac -cp "lib/*" -d out $(find src -name "*.java")
- **Run Automated QA Test Suite:** python scripts/run_qa.py
- **Launch Desktop Application:** java -cp "lib/*:out" Main